

Web Engineering (CSC-324)

Instructor: Syed Nouman Ali Shah

Assignment: Assignment 5

Max Marks: 50

Date: November 6, 2025,

Submission Date: November 15, 2025

Assignment Theme

“Product Inventory, Sales & Purchase Mini-System”

Context: You will build a small Inventory Management + Sales & Purchase backend and a simple React frontend using Spring Boot + Spring Data JPA + REST.

The system will manage:

- Products & Product Details
- Customers & Suppliers
- Sales & Purchase transactions

This assignment extends the concepts of IoC/DI and now focuses on entities, repositories, business rules, and live integration with a React client.

Target Domain Model (Minimum Requirements)

You must implement at least the following entities:

1. Product
 - id (Long, PK)
 - code (String, unique, e.g., “P-1001”)
 - name (String, not null)
 - unitPrice (BigDecimal / double, ≥ 0)
 - currentStock (int, ≥ 0)
 - active (boolean – can this product be sold?)
2. ProductDetail (One-to-One with Product)
 - id (Long, PK)
 - product (OneToOne $\rightarrow$ Product)
 - brand (String)
 - category (String, e.g., “Grocery”, “Electronics”)
 - description (String)
 - minStockLevel (int, default 0)
 - taxRate (double, percentage, e.g., 17.0)

3. Customer
 - id (Long, PK)
 - name (String, not null)
 - email (String, unique, valid email)
 - phone (String)
 - address (String)
 - blocked (boolean – if true, system should not allow new sales)
4. Supplier
 - id (Long, PK)
 - name (String, not null)
 - email (String, unique, valid email)
 - phone (String)
 - companyName (String, optional)
 - address (String)
 - active (boolean – if false, system should not allow purchases from this supplier)
5. Sale (Sales transaction for a single product)
 - id (Long, PK)
 - product (Many-to-One → Product)
 - customer (Many-to-One → Customer)
 - date (LocalDateTime)
 - quantity (int, > 0)
 - unitPrice (double or BigDecimal)
 - totalPrice (double or BigDecimal)
 - status (String: “CONFIRMED”, “CANCELLED”)
6. Purchase (Purchase transaction for a single product)
 - id (Long, PK)
 - product (Many-to-One → Product)
 - supplier (Many-to-One → Supplier)
 - date (LocalDateTime)
 - quantity (int, > 0)
 - unitCost (double or BigDecimal)
 - totalCost (double or BigDecimal)
 - status (String: “RECEIVED”, “CANCELLED”)

 Note: To keep the assignment manageable, each Sale and Purchase links to one product only. In real systems we use line-items, but here we keep it simple.

Part 1 – Spring Data JPA Domain & Repositories (15 marks)

Task 1: Entity Design & Validation

Objective: Create proper entities and map relationships using JPA.

Instructions:

1. Implement the six entities:
 - Product, ProductDetail, Customer, Supplier, Sale, Purchase.
2. Use appropriate JPA annotations:
 - @Entity, @Id, @GeneratedValue
 - @OneToOne, @OneToMany, @ManyToOne, @JoinColumn
3. Map relationships:
 - Product ↔ ProductDetail → One-to-One.
 - Product ↔ Sale / Purchase → One-to-Many (from Product side, optional).
 - Customer ↔ Sale → One-to-Many (optional).
 - Supplier ↔ Purchase → One-to-Many (optional).
4. Add validation:
 - @NotBlank / @NotNull for mandatory fields (name, code, email, etc.).
 - @Email for email fields.
 - @Min(0) for currentStock, unitPrice, unitCost, etc.
 - @Min(1) for quantity in sale/purchase.
5. Use Lombok (@Data, @NoArgsConstructor, @AllArgsConstructor) if allowed.

 Learning Outcome: Entity design, relationships, and validation in a realistic business domain.

Task 2: JPA Repositories & Sample Data

Objective: Use Spring Data JPA repositories to interact with the DB.

Instructions:

1. Create repository interfaces:
 - ProductRepository extends JpaRepository<Product, Long>
 - ProductDetailRepository extends JpaRepository<ProductDetail, Long>
 - CustomerRepository extends JpaRepository<Customer, Long>
 - SupplierRepository extends JpaRepository<Supplier, Long>
 - SaleRepository extends JpaRepository<Sale, Long>
 - PurchaseRepository extends JpaRepository<Purchase, Long>
2. Add a few derived query methods, e.g.:
 - List findByCustomerId(Long customerId);
 - List findByProductId(Long productId);
 - List findBySupplierId(Long supplierId);
 - List findByActiveTrue();
 - List findByCurrentStockLessThan(int stockLimit);
3. In a CommandLineRunner or DataLoader:
 - Insert at least:
 - 5 products with corresponding product details.
 - 3–5 customers.
 - 3–5 suppliers.
 - Create a few sample purchases to give products some initial stock.

 Learning Outcome: Students define repository interfaces, seed DB, and prepare realistic test data.

Part 2 – RESTful APIs for Inventory, Sales & Purchase (20 marks)

Task 3: CRUD Endpoints for Master Data

Objective: Expose REST endpoints for Products, Customers, Suppliers.

Controllers (suggested base paths):

- ProductController – /api/products
- CustomerController – /api/customers
- SupplierController – /api/suppliers

Minimum operations per master entity:

- GET /api/products – list all products
- GET /api/products/{id} – single product
- POST /api/products – add new product (+ productDetail)
- PUT /api/products/{id} – update product (+ productDetail)
- DELETE /api/products/{id} – delete or soft-delete (optional)
- Similarly for customers and suppliers.

Requirements:

- Use DTOs for create/update if you prefer cleaner APIs (recommended).
- Return proper HTTP statuses:
 - 201 CREATED for successful POST
 - 200 OK for successful GET/PUT
 - 204 NO CONTENT for successful DELETE
 - 404 NOT FOUND if entity not found

 Learning Outcome: Students design standard CRUD APIs for master tables.

Task 4: Sales & Purchase Endpoints with Business Rules

Objective: Implement sales and purchase logic with inventory updates and validation.

You must enforce at least the following **business rules**:

1. **Sufficient Stock for Sale**
 - When creating a **Sale**:
 - quantity must be > 0.

- `product.currentStock` must be $\geq$ **quantity**.
 - If not, return 400 BAD REQUEST with a meaningful error (e.g. “Not enough stock”).
2. **No Sales for Inactive or Out-of-Stock Product**
 - You cannot create a sale if:
 - `product.active == false`, OR
 - `product.currentStock == 0`.
 - Return 400 BAD REQUEST with a relevant message.
 3. **No Sales for Blocked Customers**
 - If `customer.blocked == true`, block the sale.
 - Return 400 BAD REQUEST with message like: “Customer is blocked. Cannot create sale.”
 4. **Purchases Update Stock, Supplier Must Be Active**
 - For **Purchase** creation:
 - `quantity` must be > 0 .
 - `supplier.active` must be true.
 - On successful purchase: **increase** `product.currentStock` by `quantity`.
 5. **Sales Decrease Stock**
 - On successful sale (status “CONFIRMED”):
 - **decrease** `product.currentStock` by `quantity`.

Note: You can implement more rules if you wish (e.g., warning if stock falls below `minStockLevel`).

Controllers (suggested base paths):

- SaleController – `/api/sales`
- PurchaseController – `/api/purchases`

Endpoints:

1. Create Sale
 - POST `/api/sales`
 - Request DTO: `SaleRequestDTO { Long productId; Long customerId; int quantity; Double unitPrice; }`
 - Backend:
 - Load product & customer.
 - Apply business rules.
 - Calculate `totalPrice = quantity * unitPrice`.
 - Set status = “CONFIRMED” by default.
 - Update `product.currentStock`.
2. Create Purchase
 - POST `/api/purchases`

- Request DTO: PurchaseRequestDTO { Long productId; Long supplierId; int quantity; Double unitCost; }
- Backend:
 - Load product & supplier.
 - Apply rules.
 - Calculate totalCost = quantity * unitCost.
 - Set status = "RECEIVED".
 - Update product.currentStock (increase).

3. List APIs:

- GET /api/sales – list all sales.
- GET /api/purchases – list all purchases.
- GET /api/customers/{id}/sales – list all sales for a customer.
- GET /api/suppliers/{id}/purchases – list all purchases from a supplier.
- GET /api/products/{id}/sales – sales for a product (optional).
- GET /api/products/{id}/purchases – purchases for a product (optional).

 Learning Outcome: Implementing transactional logic, updating stock, and enforcing domain rules.

Task 5: Centralized Error Handling

Objective: Improve API quality using @RestControllerAdvice.

Instructions:

1. Define custom exceptions:
 - ProductNotFoundException
 - CustomerNotFoundException
 - SupplierNotFoundException
 - BusinessRuleViolationException
2. Create an exception handler class:
 - @RestControllerAdvice
 - Handle above exceptions and return JSON, e.g.:

```
{
  "timestamp": "2025-11-15T10:00:00",
  "status": 400,
  "error": "Business rule violated",
  "message": "Not enough stock to complete this sale"
}
```

3. Ensure all services throw these exceptions where appropriate.

✅ Learning Outcome: Understanding error handling in REST APIs and returning consistent JSON responses.

Part 3 – React Frontend: Inventory & Sales UI (10 marks)

Task 6: Product Listing + Sales & Purchase Forms

Objective: Build a React UI to manage sales and purchases.

Instructions:

1. Create a ProductList component:
 - On mount, call GET /api/products.
 - Display results in a table (or cards) with columns:
 - Code, Name, Unit Price, Current Stock, Active.
 - Add a search/filter input (by name or code, client-side filter is fine).
2. For each product row/card:
 - Show two buttons:
 - Sell – opens a form/modal for creating a Sale.
 - Purchase – opens a form/modal for creating a Purchase.
3. Sale Form:

- Dropdown to select Customer populated from GET /api/customers.
- Input for quantity.
- Optional input for unitPrice (default to product.unitPrice).
- On submit → POST /api/sales.
- Show success message or error from backend (e.g., “Not enough stock”).

4. Purchase Form:

- Dropdown to select Supplier from GET /api/suppliers.
- Input for quantity.
- Input for unitCost.
- On submit → POST /api/purchases.
- Show success/error messages.

✅ Learning Outcome: Integrating React forms with REST API and handling responses.

Task 7: Customer Sales Dashboard

Objective: Show sales history per customer.

Instructions:

1. Create CustomerDashboard component:
 - Dropdown to select a Customer (populate from /api/customers).
 - After selection, call GET /api/customers/{id}/sales.
2. Display sales in a table:
 - Columns: Date, Product Code, Product Name, Quantity, Unit Price, Total Price, Status.
3. Add simple filters:
 - By status: dropdown (ALL / CONFIRMED / CANCELLED).
 - Optional: date range filter (client-side).

✅ Learning Outcome: Building simple reporting/dashboard pages with data from multiple entities.

Part 4 – Optional Bonus (Up to 5 Extra Marks)

Any one or more of these:

1. Low Stock Alert API
 - Endpoint: GET /api/products/low-stock?limit=5
 - Returns products where currentStock < minStockLevel.
 - Show them in React with a warning icon.
2. H2 / MySQL Profile
 - Use Spring profiles (dev, prod) to switch between:
 - H2 (dev)
 - MySQL (prod)
 - Add /api/envinfo endpoint showing active profile and DB type.
3. Very Basic Security
 - Add form of simple auth:
 - Only logged-in user can create Sale and Purchase.
 - GET endpoints public.
 - Can be simple in-memory users.

Submission Deliverables

Deliverable	Description
Backend (Spring Boot Project)	Zipped <code>src</code> folder with all entities, repositories, services, controllers, exception handler, and data loader. Include <code>application.properties</code> / <code>application.yml</code> .
Frontend (React App)	Zipped React project (<code>src</code> folder is okay) showing Product list, Sell/Purchase forms, and Customer Dashboard.
Short Report (2–4 pages)	PDF/Word explaining: (a) Your domain model and relationships, (b) Business rules enforced and where, (c) Screenshots of successful sale/purchase and at least one error case, (d) Any bonus work.

Marking Breakdown (Suggested)

Part 1 – Domain & Repositories: 30 marks

Part 2 – REST APIs & Business Rules: 40 marks

Part 3 – React Frontend: 20 marks

Code Quality & Report: 5 marks

Bonus: up to +5 marks